Prototipo de modelo de reidentificación de gatos mediante embeddings, como apoyo a la búsqueda de mascotas perdidas en Cali

Dillan Steven Molina Castro
Docente: José Luis Paniagua y Jesús Alfonso López Sotelo
Universidad Autónoma de Occidente

Introducción

En los últimos años, la rápida integración de la Inteligencia Artificial en la vida cotidiana ha transformado la manera en que nos relacionamos con el entorno. Concebida como una herramienta clave, la IA permite impulsar proyectos interdisciplinarios orientados al avance colectivo y a la solución de complejas problemáticas sociales.

En este contexto, la relación entre el ser humano y los animales de compañía, especialmente perros y gatos ha sufrido una profunda transformación. Las mascotas han dejado de ser percibidas como guardianes, ayudantes de caza u objetos para convertirse en miembros integrantes del núcleo familiar, modificando nuestras dinámicas emocionales y comportamentales.

Sin embargo, la pérdida de un animal de compañía representa una gran problemática. De acuerdo con el estudio de Weiss et al. (2012), citado por Lost Pet Research, entre el 11 % y el 16 % de los perros y entre el 12 % y el 18 % de los gatos se pierden al menos una vez en un período de cinco años. De estos, solo entre el 1.5 % y el 4.5 % de los perros y entre el 1 % y el 5 % de los gatos son encontrados.

Esta situación impacta de manera directa a los albergues y fundaciones, muchos de los cuales carecen de sistemas centralizados para gestionar y organizar la información. En Colombia, la Secretaría de Salud estima que existen entre 2 y 3 millones de animales en condición de calle, una cifra que satura la capacidad de los hogares de paso. Asimismo, emergencias y desastres naturales, como el reciente sismo del 10 de agosto registrado en el occidente del país, agravan el escenario, incrementando drásticamente el reporte de animales extraviados en redes sociales.

Ante esta realidad, surge la necesidad apremiante de centralizar la información mediante sistemas automatizados de identificación y búsqueda de mascotas.

Planteamiento del problema

Hoy, para encontrar mascotas perdidas, la gente usa publicaciones en redes sociales y grupos de mensajes como Facebook y WhatsApp. Esto tiene un problema. Las publicaciones las ven pocas personas y se pierden entre cientos de publicaciones nuevas cada día. Además, no siempre se puede evitar que una mascota se pierda. Basta una puerta abierta o un dispositivo de localización sin batería. Por eso el problema no es solo que la mascota se pierda, sino que no hay un lugar donde toda esa información esté junta y se pueda comparar.

Para entender mejor la situación se definieron tres grupos. El Grupo A son personas que han perdido mascotas. El Grupo B son personas que tienen mascotas actualmente. El Grupo C son colaboradores que apoyan a fundaciones o rescatistas en la búsqueda y el reporte de animales. Con los grupos A y B se hicieron cuatro entrevistas, dos por escrito y dos por videollamada. Los participantes, cada uno por su lado, mencionaron que la información está dispersa, lo que apoya la idea del proyecto. Con el Grupo C se aplicó un cuestionario sobre cómo es la búsqueda en el día a día.

Al empezar a desarrollar el prototipo, el problema se volvió más concreto. Ya no basta con juntar la información, también importa que las fotos sirvan para reconocer al animal. El cuestionario lo respondió una voluntaria de Cali que colabora con fundaciones y rescatistas. Esto fue lo que contó:

- Dónde busca hoy. Su primer paso es revisar las publicaciones de “mascotas perdidas” para ver si alguien está buscando al animal. Los grupos de Facebook son lo que mejores resultados le ha dado. Aun así, dice que las publicaciones funcionan a veces y se pierden rápido entre las nuevas.
- Confirmar al dueño es difícil. Calificó con 7 sobre 10 qué tan difícil es comprobar que quien reclama al animal es su dueño. Lo más frustrante de esa verificación, según ella, son las fotos.
- Lo que más le limita. Mencionó la falta de tiempo, la cantidad de mensajes en redes y la dificultad para diferenciar animales con pelaje parecido.
- El problema de la foto. Lo más común al fotografiar a los animales es que se mueven mucho y las fotos salen borrosas.

Todo esto muestra que un sistema que compare fotos va a recibir imágenes de calidad desigual, borrosas o con partes del cuerpo tapadas, y gatos distintos con rasgos parecidos.

Propuesta de solución

Este proyecto busca ayudar a encontrar mascotas perdidas a partir de una foto. En esta entrega se desarrolla un prototipo para gatos. La definición completa del proyecto está en la primera entrega.

Datos principales:

Para el entrenamiento se usó el dataset **Cat Individual Images,** creado por Tzu-Yuan Lin y disponible en https://creativecommons.org/licenses/by/4.0/, bajo licencia CC BY 4.0. Contiene 13,536 imágenes de 509 gatos, tomadas con cámaras digitales y teléfonos móviles, en su mayoría en albergues públicos y privados de Taipéi y Nueva Taipéi (Taiwán), y el resto obtenidas de redes sociales. La resolución varía entre 195 × 261 y 4608 × 3453 píxeles. Tras la depuración descrita en la primera entrega, el conjunto de partida fue de 13,106 imágenes, y el EDA completo se encuentra en dicha entrega. Las imágenes fueron modificadas para este trabajo (recorte con YOLO y redimensionamiento a 300 × 300 píxeles).

El sistema funciona en dos fases.

Fase 1 Detección, Filtrado y Preprocesamiento de Datos:

Un modelo de detección de objetos preentrenado basado en YOLO26m, cuya tarea principal es revisar la imagen ingresada. Si el modelo detecta exactamente un gato, procede a recortarlo, si no encuentra ningún felino o detecta más de uno, la foto se descarta de inmediato.

Este pipeline se complementó con dos validaciones clave: una prueba de generalización utilizando un subconjunto de COCO 2017 para medir la precisión del detector, y una segunda validación sobre una muestra de 300 imágenes seleccionadas de un total de 9,000 registros provenientes de https://www.kaggle.com/datasets/crawford/cat-dataset. Dado que este conjunto carecía de los archivos de etiquetas originales, dichas muestras fueron procesadas y anotadas manualmente

Esta fase tiene los propósitos:

1. Validar y limpiar la presencia de los animales dentro de las imágenes del dataset durante el entrenamiento.

2. Servir como una herramienta clave en la fase de despliegue en producción para evitar errores ante imágenes irrelevantes o mal cargadas por los usuarios.

El pipeline inició con un conjunto de datos bruto compuesto inicialmente por 13,106 imágenes. Tras la validación y limpieza inicial con el modelo de detección (YOLO), se descartaron registros sin presencia de felinos o con problemas de integridad, contabilizando un total de 12,872 imágenes válidas.

Posteriormente, para la localización y recorte, se identificaron 10,520 imágenes validas. La diferencia de 2,352 imágenes corresponde a imágenes que contenían múltiples felinos (mínimo dos en la misma), los cuales se descartaron.

Este conjunto depurado se dividió en:

- **Conjunto de Entrenamiento:** 8,395 registros.
- **Conjunto de Testeo:** 2,125 registros.

Para el recorte, se emplearon las coordenadas de las cajas delimitadoras obtenidas por el modelo, añadiendo un padding de 1.1 para asegurar que no se pierdan proporciones, ni los bordes del animal respecto a la imagen original. El objetivo es que el modelo de REID procese únicamente la información de interés del felino y descarte el fondo, reduciendo así la cantidad de ruido.

Finalmente, las imágenes recortadas pasan por un proceso de preprocesamiento donde se redimensionan a una resolución fija de 300x300 píxeles, se escalan las imágenes manteniendo su proporción original y rellenando los espacios sobrantes con un fondo negro Esto permite estandarizar las entradas y conservar las características visuales y patrones del pelaje necesarios para la reidentificación, al tiempo que optimiza y acelera los procesos de entrenamiento y testeo del modelo de Re-ID.

Fase 2 Extracción de Características y Reidentificación.

Las imágenes de los felinos validadas, recortadas y redimensionadas ingresan al módulo de reidentificación basado en aprendizaje métrico profundo (Res-Net 18). La selección de esta arquitectura responde a un balance óptimo entre capacidad de abstracción y eficiencia computacional. Al contar con un volumen aproximado de 11.7 millones parámetros en comparación con arquitecturas más profundas como ResNet-50, se garantiza un tiempo de inferencia rápido, menor consumo de memoria y un riesgo menor de sobreajuste durante el entrenamiento de los embeddings de similitud.

1. Configuración y Arquitectura del Modelo

Se uso una red neuronal convolucional ResNet-18 con pesos preentrenados de ImageNet como backbone. Para adaptarla a reidentificación, se modificó la capa final totalmente conectada (FC) con el fin de proyectar las características espaciales en un espacio vectorial de 512 dimensiones, conocido como embeddings (Una representación numérica compacta en forma de vector que traduce las características visuales complejas del felino como patrones de pelaje y rasgos faciales en coordenadas matemáticas donde los gatos idénticos quedan muy cerca entre sí y los diferentes se alejan)

- Dimensión del Embedding: 512 valores normalizados mediante norma euclidiana, asegurando que todos los vectores residan en el espacio unitaria para simplificar y optimizar el cálculo de distancias por similitud.
- Preprocesamiento: Las imágenes se convirtieron a tensores y se normalizaron utilizando las medias y desviaciones estándar de ImageNet (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]).

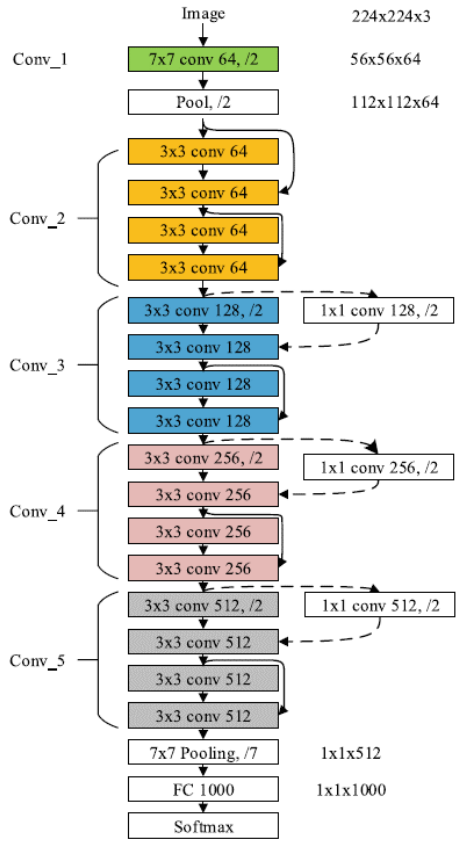


Figura 5: Diagrama de la arquitectura de la red utilizada como base para el procesamiento y extracción de vectores de características.

Para la extracción de vectores de características se empleó una arquitectura basada en Redes Residuales. A diferencia de las redes convolucionales tradicionales, esta arquitectura introduce conexiones de salto las cuales permiten que la entrada original de un bloque eluda capas intermedias y se sume directamente a su salida.

Este mecanismo resuelve el problema del desvanecimiento del gradiente en redes profundas y asegura que la función de activación ReLU no se aplique de forma indiscriminada en cada paso, sino después de dicha suma. De este modo, las capas aprenden únicamente los cambios residuales necesarios, garantizando un flujo de información estable y una representación más robusta para el sistema de reconocimiento métrico.

2. Hiperparametros y Entrenamiento

El entrenamiento se estructuró para optimizar el espacio métrico mediante tripletas (Anchor, Positive, Negative), garantizando que las distancias entre fotos de un mismo gato se minimicen mientras que las de diferentes individuos se maximicen:

- Hardware y Entorno: El proceso de entrenamiento se ejecutó en Google Colab con aceleración por GPU, permitiendo un procesamiento eficiente de los tensores y reduciendo los tiempos de cómputo en la extracción de características.

- **Generación de Tripletas:** Para el entrenamiento métrico se estructuraron un total de **3,702 tripletas para entrenamiento** y **786 tripletas para validación** (Anchor, Positive, Negative), orientadas a minimizar la distancia entre fotos del mismo individuo y maximizarla entre diferentes ejemplares.

- Semilla de Reproducibilidad: Se fijó una semilla aleatoria (4) en todos los procesos estocásticos (como la división de datos y el muestreo de tripletas) para garantizar la total reproducibilidad de los experimentos.

- Estrategia de Entrenamiento: Se entrenó la red de manera completa actualizando todos los pesos del modelo, permitiendo que todo el extractor de características se adaptara específicamente al dominio visual de los felinos.

- Criterio de Guardado del Modelo: Los pesos óptimos del modelo se almacenaron dinámicamente en la época 7, punto identificado en la figura 2 de pérdida donde se alcanza una estabilidad óptima antes de que el error de validación comience a divergir.

- Función de Pérdida: TripletMarginLoss configurada con un margen de 0.5 y norma L_2 (p = 2.0)

$$L(A, P, N) = \max(\|f(A) - f(P)\|_2 - \|f(A) - f(N)\|_2 + \text{margin}, 0)$$

La función evalúa simultáneamente las tres imágenes de la tripleta para medir qué tan bien está separando el modelo las identidades:

1. **Calcula las distancias:** Mide con la norma euclidiana cuán lejos está el ancla del positivo ($d(A, P)$) y cuán lejos está el ancla del negativo ($d(A, N)$).
2. **Compara con el margen:** Resta ambas distancias y le suma el margen (0.5).
    - Si el gato negativo está lo suficientemente lejos, el resultado de esa resta da un número negativo o cero.
    - La función se encarga de que, si el modelo funciono, la penalización sea exactamente cero.
3. **Penaliza el error:** Si el gato negativo se acerca demasiado al ancla, la resta da un valor positivo, activando un castigo proporcional que obliga a la red a ajustar sus pesos para separar los vectores en el siguiente ciclo de entrenamiento.

- Optimizador: Algoritmo Adam con una tasa de aprendizaje de 0.0001 para asegurar una convergencia estable.

- Tamaño de Lote: Lotes de 16 muestras para la eficiencia en el uso de vram y la estabilidad del gradiente.

- División del Dataset: Mediante la función train test Split utilizando identificadores únicos de cada animal, para evitar filtración de datos entre particiones de entrenamiento y prueba. Adicionalmente, se implementó un guardado dinámico por época que almacena automáticamente los pesos del modelo únicamente cuando el error promedio registra un mínimo histórico.

3. Evaluación y Rendimiento del Sistema

Para la fase de validación experimental sobre el conjunto de testeo, los 2125 registros totales se distribuyeron en 1929 registros en la galería y 196 registros de

consulta, tras aplicar el criterio de selección en el que cada individuo cuenta con un mínimo de una imagen base de referencia y un máximo de dos muestras de consulta. Este proceso arrojó las siguientes métricas de rendimiento:

- Rank-1: 92.86%
- Rank-5: 96.94%
- Rank-10: 97.45%
- mAP (Mean Average Precision): 75.27%

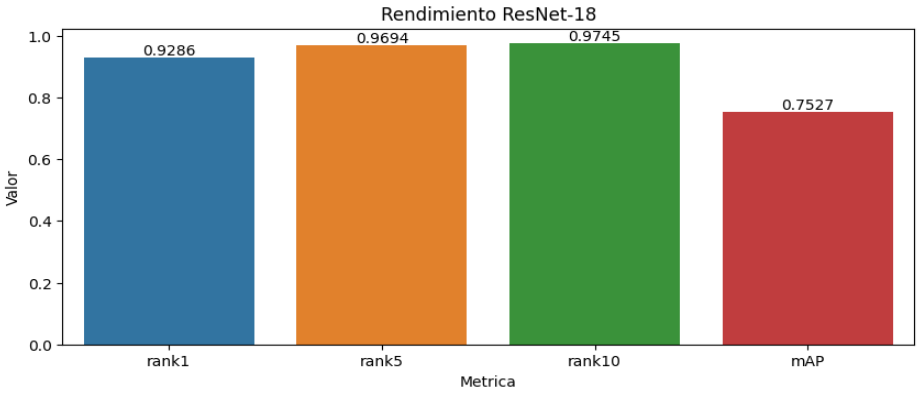


Figura 1: Desempeño del modelo de reidentificación felina ResNet-18 según métricas de evaluación.

Estos resultados confirman la alta efectividad del modelo en el espacio de características evaluado, logrando que en casi 9 de cada 10 consultas la primera coincidencia devuelta corresponda exactamente al mismo felino buscado.

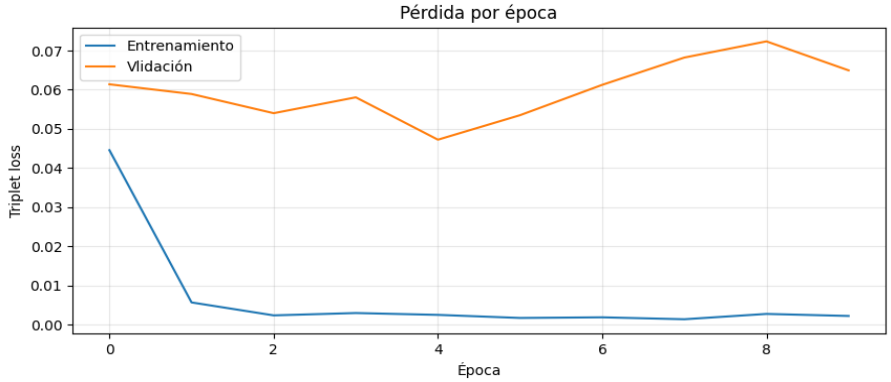


Figura 2: Evolución de la pérdida (Triplet Loss) por época durante el entrenamiento y la validación del modelo, donde se observa la convergencia del conjunto de entrenamiento y el comportamiento del error de validación a lo largo de las 10 épocas.

Según la figura 2, el conjunto de entrenamiento converge rápidamente. Aunque la curva de validación muestra una leve tendencia al alza, al analizar la escala del eje vertical (10^{-2}, con variaciones entre 0.04 y 0.07) se descarta un overfitting severo.

La diferencia es milimétrica, lo que confirma que el modelo mantiene un rendimiento estable y una buena capacidad de generalización.

4. Discusión y Análisis Crítico de Resultados

A pesar de los resultados obtenidos, es importante señalar que la evaluación actual podría presentar un ligero sesgo positivo. Porque varias de las imágenes de un mismo individuo en el dataset comparten condiciones de iluminación, ángulo y entorno muy similares, lo que simplifica la tarea del extractor de características.

Adicionalmente, mediante el análisis del rendimiento, se identificó un sesgo morfológico en el modelo de detección (YOLO) frente a felinos sin pelaje (por ejemplo, razas tipo Sphynx). La ausencia de texturas de pelaje y las particularidades anatómicas de estos ejemplares provocan caídas en la confianza o fallos en la generación de las cajas delimitadoras, evidenciando una subrepresentación de esta morfología en los datos de entrenamiento originales.

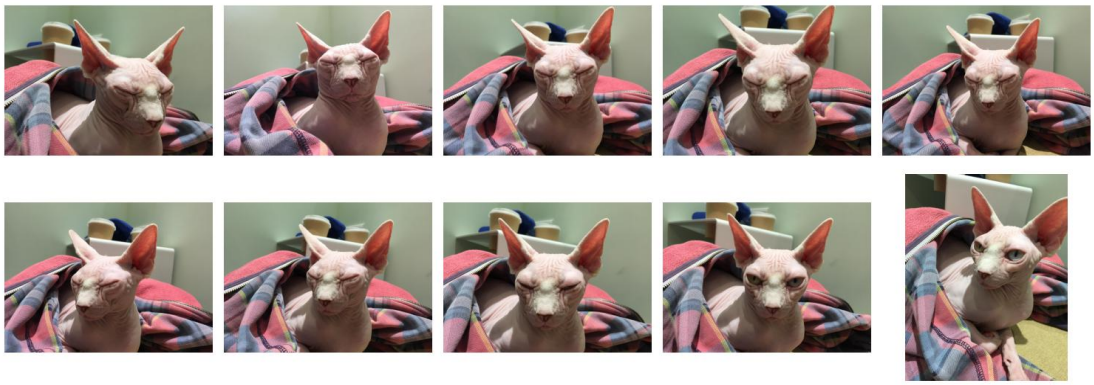


Figura 3: Ejemplos de imágenes del conjunto de datos de un gato de raza Sphynx que el modelo no logró detectar, posiblemente debido a la textura de su piel y características morfológicas particulares.



Figura 4: Instancias adicionales de un individuo Sphynx no detectadas, evidenciando los casos en los que las variaciones de la raza superan la capacidad de generalización del detector.

Para mitigar este sesgo y elevar la capacidad de generalización del sistema en escenarios reales y descontrolados, se proponen las siguientes líneas de trabajo a futuro:

- **Minería de Tripletas Difíciles:** Dado que en el entrenamiento actual las tripletas se formaron de manera aleatoria, implementar una estrategia de Hard Negative Mining forzará al modelo a comparar el gato objetivo con las muestras negativas más parecidas visualmente dentro del lote. Esto obligará a la red a aprender diferencias sutiles en los patrones del pelaje y rasgos faciales únicos en lugar de apoyarse en características globales evidentes.
- **Validación Cruzada:** Reorganizar el esquema de prueba mediante particiones por pliegues para garantizar que ningún subconjunto de imágenes similares quede expuesto simultáneamente en entrenamiento y validación, asegurando un cálculo de mAP aún más riguroso.
- **Aumento de Datos y Robustez ante Desenfoque:** Incorporar técnicas avanzadas de aumento de datos enfocadas en simular condiciones adversas tales como desenfoque por movimiento, ruido de sensor y variaciones de iluminación, asegurando que tanto el modelo de detección (YOLO) como el extractor de características mantengan su estabilidad operativa ante imágenes de menor calidad capturadas en entornos reales.
- **Evaluación con Arquitecturas Alternativas de Re-ID:** Contrastar el rendimiento de la columna vertebral actual (ResNet-18) frente a modelos más profundos o especializados en reidentificación (como ResNet-50, EfficientNet), con el fin de analizar el equilibrio entre la ganancia en precisión y el costo computacional.

Bibliografía:

- PyTorch. (s. f.). *torch.nn.TripletMarginLoss — PyTorch 2.14 documentation*. Recuperado de https://docs.pytorch.org/docs/2.14/generated/torch.nn.TripletMarginLoss.html

- Autor del video. (s. f.). *[Título del video de YouTube]* [Video]. YouTube. https://youtu.be/ZoZHd0Zm3RY

- Nokia. (s. f.). *torch-ngp: NeRF implementation in PyTorch* [Repositorio de GitHub]. GitHub. https://github.com/nokia/ashawkey-torch-ngp

- Roboflow. (s. f.). *ResNet-18: Overview, Architecture, and Code*. Roboflow Blog. https://roboflow.com/blog/resnet-18

- Kaggle. (s. f.). *COCO 2017 Dataset*. Kaggle. https://www.kaggle.com/datasets/awsaf49/coco-2017-dataset

- Ultralytics. (s. f.). *COCO Dataset - Ultralytics YOLO Docs*. https://docs.ultralytics.com/datasets/detect/coco

- Qdrant. (s. f.). *Understanding Triplet Loss and its application*. Qdrant Tech. https://qdrant.tech/articles/triplet-loss/

- ResearchGate. (s. f.). *Illustration of the architecture of ResNet-18 model* [Figura]. ResearchGate. https://www.researchgate.net/figure/Fig-2-An-illustration-of-the-architecture-of-ResNet-18-model_fig1_383708698

- Ultralytics. (s. f.). *Desvanecimiento del gradiente (Vanishing Gradient)*. Glosario Ultralytics. https://www.ultralytics.com/es/glossary/vanishing-gradient